Dynamic Speculative Agent Planning

用在线预测的动态投机深度控制延迟—成本权衡



Yilin Guan et al. • arXiv:2509.01920

汇报人：张帅 | Agent 动作投机 Survey | 2026.08

为什么固定投机步数 k 不够?

背景 / 核心问题

- Agent 规划串行且延迟高；快 A 可草拟，慢 T 负责权威验证。
- k 太小：并行窗口没吃满； k 太大：失配后的草拟调用全部浪费。
- 同一任务的不同阶段难度不同，不存在始终最优的固定 k 。

主要贡献

1 Pareto 分析

将 k 作为延迟—成本的核心控制量。

2 动态 k 预测

DistilBERT 状态价值预测器在线学习连续命中长度。

3 用户可控

expectile 或 offset 将运行点在“更快—更省”间移动。

最终轨迹始终由 Target 决定；动态预测只改变提前执行多远。

固定 k：加速来自并行，浪费来自失配之后

1 A 草拟 k 步

快 Approximation Agent 连续生成未来规划。

2 T 并行验证

慢 Target Agent 从同一状态生成权威步骤。

3 首个 mismatch

取消失配点之后所有未确认线程。

4 跟随 T 恢复

从 Target 的正确步骤继续，保持轨迹一致。

k < m

冗余少，但留下未利用并行窗口。

k ≈ m

接近实际连续命中长度，收益最好。

k > m

延迟不再下降， token/API 冗余上升。

DSP： 在每个投机 episode 中动态决定 k



A/T 主执行线

决定权威轨迹

Predictor inference

与 A 当前步并行

Trainer

不阻塞当前任务

预测“还能连续命中几步”：状态价值建模

状态与回报

s_t 是已验证轨迹； G_t 是到首次 mismatch 之前的实际连续命中步数。

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \qquad G_t = \sum_{i=t}^{T-1} \gamma^{i-t} R(s_i, a_i)$$

TD / λ-return 目标

DistilBERT 只回归 Vθ(s)，不负责生成 Agent 动作。

$$L_\theta = \mathbb{E}\left[(G_t^\lambda - V_\theta(s_t))^2\right]$$

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_t$$

λ = 0

更接近 1-step TD，方差小、偏差大。

0 < λ < 1

在短期 bootstrap 与完整回报之间做权衡。

λ = 1, γ = 1

G_t = k，退化为在线监督回归可命中步数。

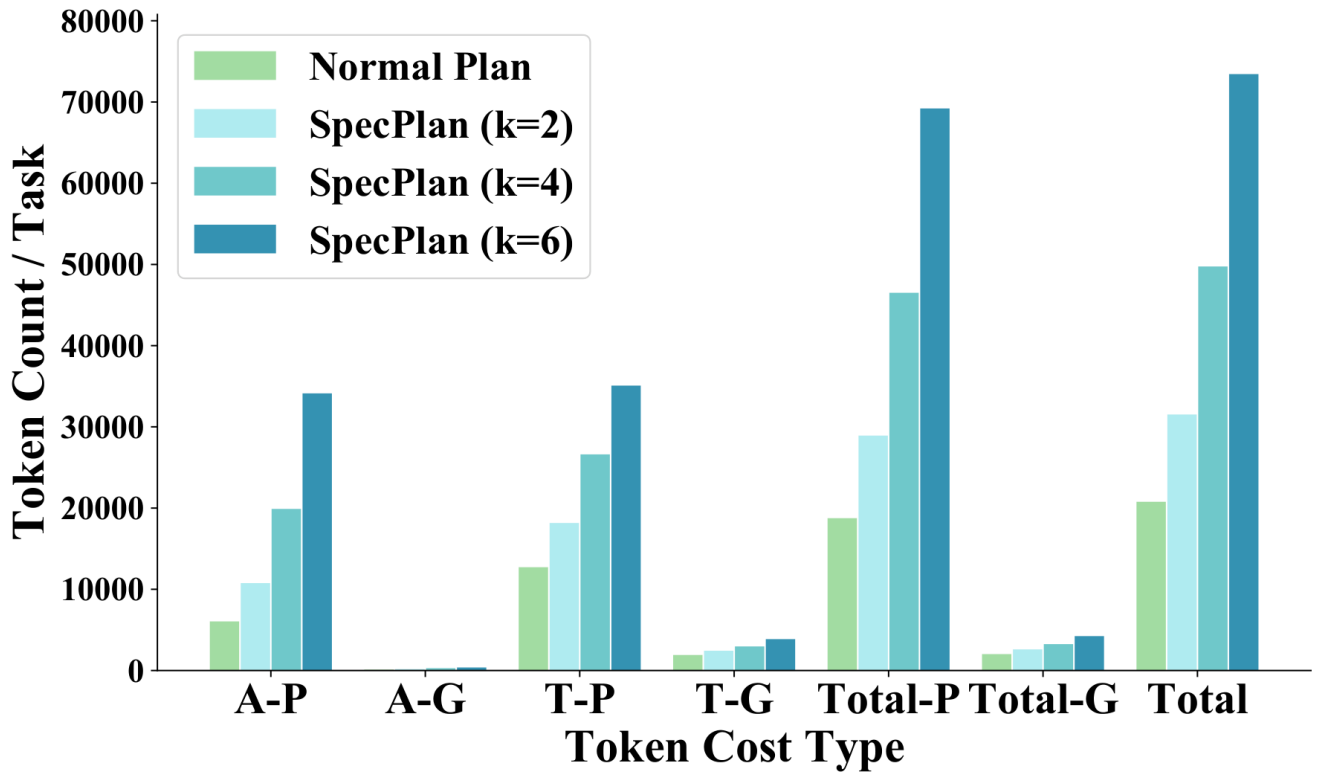
一个预测器，两种“更激进 / 更保守”控制方式

A | Biased value / expectile

用非对称损失让估计更偏高或偏低。
τ > 0.5： 更激进、更低延迟、更高成本。
τ < 0.5： 更保守、更少冗余。

B | Unbiased value + offset

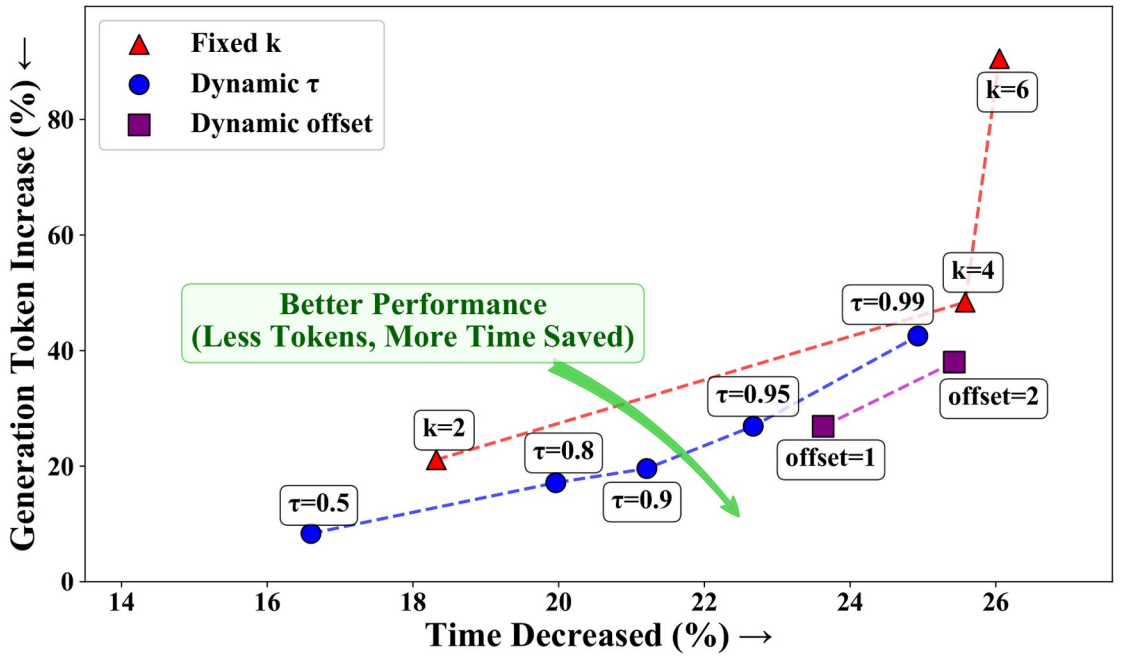
保持价值估计无偏，部署时用
k = clip(round(Vθ(s)) + β)
β > 0 向速度端移动； β < 0 向成本端移动。



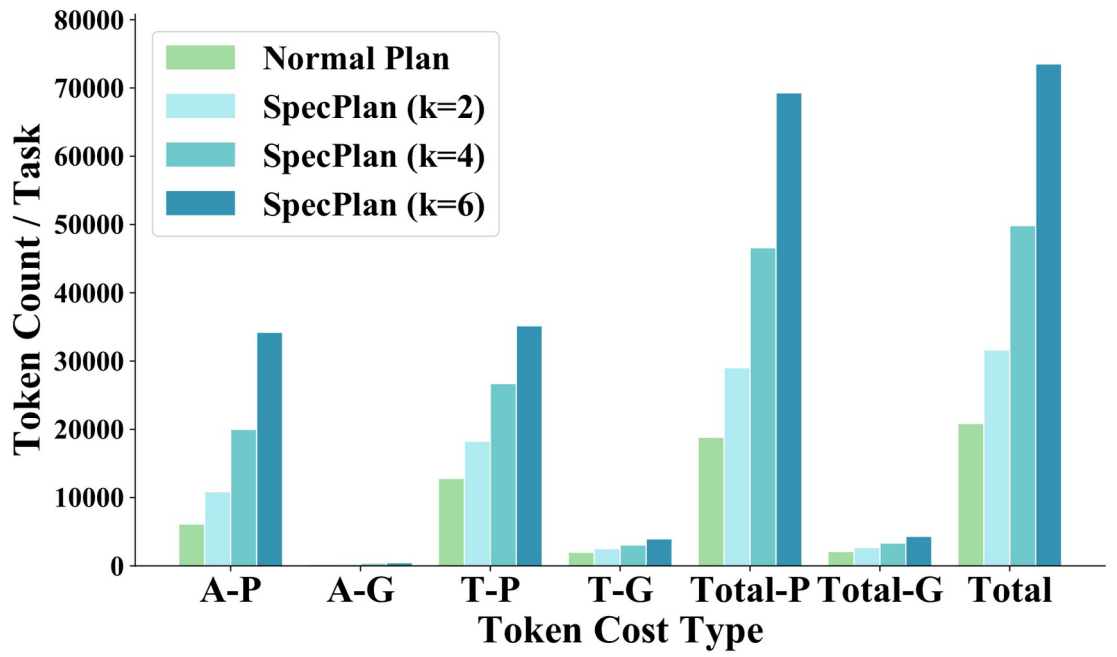
论文原图： OpenAGI ReAct cost comparison

每个 τ / β 都是 Pareto 平面上一个可配置运行点，不是准确率调参。

动态 k 追平最快 fixed-k，但减少无效调用



生成量—延迟关系



成本—延迟 Pareto 对比

≈ fastest
延迟

-30%
总成本（最多）

-60%
无效成本（最多）

实验重点是成本和延迟；质量不降由“最终跟随 Target” 保证。

DSP 解决的是“投机多远”，不是“如何预测动作”

三句话带走

01

核心对象：动态投机深度 k，匹配状态的局部可预测性。

02

方法关键：将连续命中长度写成状态价值，异步在线更新。

03

系统意义：在不改变 Target 轨迹前提下，显式配置速度—成本点。

适用边界 / 限制

- 未动态选择 Approximation Agent，仍受 A/T 速度比和一致性限制。
- 严格步骤对齐可能拒绝语义等价规划。
- 在线冷启动和跨域泛化证据仍有限。

Survey 技术链中的位置

Speculative Actions
预测—验证—提交
↓
DSP
在线动态深度 k